Davante

Desarrollo de Aplicaciones Web
Desarrollo Web Entornos Cliente

Tema 2.
Lenguajes y herramientas de programación en clientes web

Diego Rodero Pulido

Tarea 2. Navegadores, motores y tu primera página interactiva

Tema 2. Lenguajes y herramientas de programación en clientes web · Tarea individual · Calificación: 7 puntos de rúbrica + hasta 3 puntos en la defensa en clase.

Objetivo

Conocer las piezas con las que se trabaja en el lado del cliente —el navegador y sus motores, HTML, CSS y JavaScript— y dejar preparado el entorno de trabajo del curso. Al terminar la tarea debes ser capaz de:

- Identificar el motor de renderizado y el motor de JavaScript de los principales navegadores, y explicar por qué una misma página puede comportarse de forma distinta en cada uno.
- Construir un pequeño sitio de dos páginas con Bootstrap a partir de la plantilla base, con la etiqueta viewport, que se vea bien tanto en el ordenador como en el móvil.
- Integrar JavaScript desde un archivo externo para responder a lo que hace el usuario y dejar trazas en la consola del navegador.
- Distinguir qué aporta cada capa (HTML, CSS y JavaScript) y explicar y modificar tu propio código delante del profesor.

Qué vas a construir

En la carpeta `tema02` de tu repositorio de DWEC vas a crear un pequeño sitio web de **dos páginas**, una por cada una de las partes A y B. En la parte C lo documentas con evidencias propias.

Página	Parte	Contenido
`index.html`	A. Navegadores y motores	Tabla de navegadores, texto propio, ejemplo de compatibilidad y reflexión.
`interaccion.html`	B. Interacción con JavaScript	Tres botones con JavaScript y trazas en la consola.

Las dos páginas cumplen estos requisitos comunes:

- Parten de la plantilla base de Bootstrap (`plantilla-base.html`), que está en la carpeta `plantilla` del GitHub de la asignatura: https://github.com/DRodero/DWEC_2627. Llevan la etiqueta viewport y cargan el CSS y el JS de Bootstrap.

- Comparten la misma barra de navegación (`navbar`), con **tu nombre y apellidos** y un enlace a cada página.
- Se maquetan con `.container`, `.row` y `.col-*`, y en el móvil no necesitan scroll horizontal.

Parte A. Navegadores y motores (index.html)

Prepara el contenido de `index.html` a partir de los apuntes, la presentación del tema y, al menos, dos fuentes complementarias fiables. La página debe incluir:

1. Una tabla (componente `table` de Bootstrap) con Chrome, Firefox, Safari, Edge y Opera. Para cada uno: empresa que lo desarrolla, motor de renderizado, motor de JavaScript y si está basado en Chromium o no.
2. Debajo de la tabla, un texto propio de 5 a 8 líneas que explique por qué casi todos los navegadores comparten motor, cuáles se quedan fuera y qué supone eso para quien desarrolla una web.
3. Un ejemplo real de diferencia de compatibilidad: busca en caniuse.com una característica de HTML, CSS o JavaScript que no funcione igual en todos los navegadores. Indica cuál es, en qué navegador o versión falla, el enlace y la fecha de consulta. Preséntalo dentro de una `card`.
4. Una reflexión de 3 a 4 líneas: si un cliente te encarga una web, ¿en qué navegadores la probarías primero y por qué? Ten en cuenta el móvil (caso práctico 1 del tema).

Parte B. Interacción con JavaScript (interaccion.html)

1. Además de la navbar y de los botones, usa en esta página al menos un componente más de Bootstrap (por ejemplo, una `card` que agrupe los botones o un `list-group` que explique qué hace cada uno).
2. Escribe todo el JavaScript en `js/app.js` y enlázalo desde `interaccion.html` con `<script src="js/app.js"></script>` al final del `body`. En el HTML no debe quedar ningún bloque `<script>` con código.
3. Añade tres botones, cada uno con un comportamiento distinto. Cada botón llama a una función de `app.js` (por ejemplo, con el atributo `onclick`):
   a. «Saludar»: muestra un mensaje con `alert()` que incluya tu nombre.
   b. «Simular un error»: no abre ninguna ventana; escribe un mensaje de error en la consola con `console.error()`. El usuario no ve nada, pero quien desarrolla sí (recuerda el caso práctico 2, la aplicación bancaria).



   c. «¿Qué navegador soy?»: muestra el valor de `navigator.userAgent` en un `alert()` y también en la consola.

4. Toda la interacción se hace con `alert()` y con la consola; escribir dentro de la página lo veremos más adelante. Cada botón deja al menos una traza en la consola (`console.log()`, `console.warn()` o `console.error()`, según convenga). Al cargar cualquiera de las dos páginas, la consola no debe mostrar ningún error.
5. Prueba el sitio con Live Server en, al menos, dos navegadores con motores distintos (por ejemplo, Chrome y Firefox).

Parte C. Evidencias y explicación (README.md)

En el `README.md` de la carpeta explicas lo que has hecho y lo demuestras. Las capturas van en la carpeta `capturas/`, **tienen que estar hechas en tu equipo** y cada una lleva debajo una o dos líneas que expliquen qué se ve.

1. Cinco capturas propias:
   a. `index.html` en el ordenador, con tu nombre visible en la navbar.
   b. `interaccion.html` en modo dispositivo (F12 → barra de dispositivos), simulando un móvil.
   c. La consola con las trazas de los tres botones.
   d. El `alert()` del botón «¿Qué navegador soy?» en los dos navegadores que has usado.
   e. VS Code con la carpeta `tema02` abierta y Live Server en marcha.
2. «Quién hace qué»: elige uno de tus botones y explica qué parte corresponde a HTML, qué aporta Bootstrap (CSS) y qué hace JavaScript.
3. Compara los dos userAgent que has obtenido: qué partes reconoces y por qué aparecen palabras como Mozilla, AppleWebKit o Safari aunque el navegador sea otro.
4. Fuentes consultadas, con su enlace, y el apartado «Uso de IA» si corresponde.

Entrega

- Todo va en la carpeta `tema02` de tu repositorio público de DWEC, con esta estructura:

```
tema02/
├── index.html          parte A: navegadores y motores
├── interaccion.html    parte B: los tres botones
├── js/
│   └── app.js          el JavaScript de la parte B
├── capturas/           las cinco capturas de la parte C
└── README.md           parte C: explicación y fuentes
```

- Sube el trabajo en varios commits a medida que avances (tres como mínimo), con mensajes que digan qué cambia: «Tabla de navegadores», «Botones y consola»…
- En la tarea del campus virtual entregas solo la URL de la carpeta: `https://github.com/tu-usuario/tu-repositorio/tree/main/tema02`
- Extensión orientativa del README: de 300 a 600 palabras, más las capturas.
- Se corrige el último commit anterior a la fecha límite publicada en la propia tarea del campus; lo que subas después no cuenta. No se admiten entregas por otros medios.

Criterios de evaluación (rúbrica)

La tarea se califica sobre 7 puntos con la rúbrica siguiente. Los 3 puntos que faltan hasta el 10 se obtienen en la defensa en clase.

Criterio (puntuación máxima)	Insuficiente	Adecuado	Excelente
1. Las dos páginas con Bootstrap y viewport (1,5 p)	0 – 0,5. Falta una de las páginas, no parten de la plantilla base, falta el viewport o Bootstrap, no se ve tu nombre o alguna se rompe en el móvil.	0,75 – 1. Las dos páginas parten de la plantilla base con viewport y con el CSS y el JS de Bootstrap; navbar común con tu nombre y enlaces que funcionan; maqueta con container, row y col.	1,25 – 1,5. Maquetación limpia que se adapta al móvil sin scroll horizontal, navegación coherente entre las dos páginas, componentes bien elegidos y sin CSS propio innecesario.
2. Interacción con JavaScript y consola (2 p)	0 – 0,5. Faltan botones, el código está dentro del HTML o la consola muestra errores al cargar la página.	0,75 – 1,5. `app.js` externo, tres botones que funcionan como se pide y, al menos, una traza en la consola por botón.	1,75 – 2. Además, el código está ordenado y comentado, los mensajes son claros, distingue lo que ve el usuario (`alert`) de lo que ve quien desarrolla (consola) y usa `console.log`, `warn` y `error` con sentido.
3. Navegadores y motores (1,5 p)	0 – 0,5. Tabla incompleta o con errores de bulto (motores cambiados, Edge con motor propio…); texto copiado o genérico.	0,75 – 1. Tabla correcta de los cinco navegadores, texto propio y un ejemplo de compatibilidad con su enlace.	1,25 – 1,5. Explica con criterio el reparto entre Chromium, Gecko y WebKit; ejemplo contrastado en caniuse con su fecha y reflexión argumentada.
4. Evidencias propias y explicación (1,5 p)	0 – 0,5. Sin capturas, con capturas que no son de tu equipo o con un README vacío o genérico.	0,75 – 1. Las cinco capturas son propias y llevan su comentario; explica qué hace cada capa en uno de los botones.	1,25 – 1,5. Evidencias propias con tu nombre visible y bien comentadas; explica con precisión las tres capas y lo que has descubierto en los userAgent.
5. Entrega en GitHub (0,5 p)	0. No está en GitHub, la estructura es otra o todo el trabajo sube en un único commit.	0,25. Estructura de carpetas correcta y varios commits.	0,5. Al menos tres commits con mensajes que muestran el avance, README bien formateado y fuentes citadas.

Defensa en clase (hasta +3)

Los 3 puntos restantes se consiguen enseñándome la tarea en clase y demostrando que sabes hacerla. No se trata de repetir lo entregado: te pediré un cambio pequeño y lo harás en el momento, explicando lo que haces.

- **Cuándo:** en la sesión de clase que indique después del cierre del plazo. Unos cinco minutos por persona.
- **Requisito:** solo puedes defender la tarea si la has entregado en plazo. Si no la defiendes, tu nota es la de la rúbrica (máximo 7).
- **Cómo:** abres tu página desde la versión entregada, la enseñas funcionando con la consola abierta y haces en tu propio código el cambio que te pida.
- **Sin IA:** durante la defensa desactiva Copilot o cualquier autocompletado con IA. Puedes consultar los apuntes y la documentación de MDN.

Ejemplos de lo que te puedo pedir

- Añadir un cuarto botón con otro `alert()` y su traza en la consola.
- Cambiar un `alert()` por una traza en la consola, o al revés, y explicar quién ve cada cosa.
- Pasar una traza de `console.log()` a `console.warn()` o `console.error()` y enseñar la diferencia con F12.
- Quitar la etiqueta viewport y enseñar qué cambia en modo dispositivo.
- Encontrar y corregir, con ayuda de la consola, un error que yo introduzca en tu código.
- Explicar una línea concreta de tu código o una fila de tu tabla de navegadores.

Cómo se puntúa

Puntos	Qué demuestras
3	Haces el cambio sin ayuda y explicas con seguridad tu código y lo que aparece en la consola.
2	Haces el cambio con alguna duda o con una pista pequeña, y explicas correctamente lo esencial.
1	Necesitas bastante ayuda para hacer el cambio o solo sabes explicar partes sueltas de tu código.
0	No consigues hacer el cambio ni explicar tu propio código, o no te presentas.

Se pueden dar valores intermedios de medio punto.

Nota final = rúbrica (máximo 7) + defensa en clase (máximo 3).

Uso de la inteligencia artificial

Puedes apoyarte en herramientas de IA para orientarte, entender un error o revisar la redacción, pero el código y las explicaciones tienen que ser tuyos. Si las utilizas, indícalo al final del README en un apartado «Uso de IA»: qué herramienta, para qué y qué hiciste después con su respuesta. Declararlo no resta nota.

Advertencia: si se detecta que la tarea se ha generado con IA sin trabajo propio, la calificación será un 0 absoluto.

Son indicios: la falta de evidencias propias, el código con técnicas que no hemos visto y que no sabes explicar, los textos genéricos que no responden a lo que se pide, las fuentes inexistentes, subir todo el trabajo de golpe en un único commit y no saber explicar lo entregado. Una captura tomada de internet, prestada o generada cuenta como el mismo caso.

La defensa en clase sirve precisamente para comprobarlo: si en ella no eres capaz de explicar nada de lo que has entregado, no solo no sumas esos 3 puntos, sino que se considera un indicio claro de que la tarea no es tuya.